



A mechanism for scheduling multi robot intelligent warehouse system face with dynamic demand

Zhi Li¹ · Ali Vatankhah Barenji^{1,2} · Jiazhi Jiang¹ · Ray Y. Zhong³ · Gangyan Xu⁴

Received: 12 March 2018 / Accepted: 10 December 2018 / Published online: 14 December 2018
 © Springer Science+Business Media, LLC, part of Springer Nature 2018

Abstract

Given the evolutionary journey of E-commerce, there have been emerging challenges confronting warehouse logistics, including smaller shipping units, more varieties and batches, and shorter cycles. These challenges are difficult to cope when using conventional scheduling with the robotic approach. Currently, automated storage and retrieval system are becoming preferred for warehouse companies with the help of mobile robots. However, when many orders are received simultaneously, the existing scheduling approach might make unreasonable decisions, leading to delayed packaging of entire orders and reducing the performance of the warehouse. Therefore, this paper addresses this problem and proposes a novel scheduling mechanism for multi-robot and tasks allocation problems which may arise in an intelligent warehouse system. This mechanism proposes into the intelligent warehouse troubled with simultaneous multiple customer demands. The mathematical model for the system is developed by considering a multitask robot facing dynamic customer demand. The proposed model's approach is based on the particle swarm optimization heuristic. The result for this approach then compared with the genetic algorithm (GA). The simulation results demonstrate that the proposed solution is far superior to that of the GA for multi-robot scheduling and tasks allocation problems in the intelligent warehouse.

Keywords Intelligent warehousing system · Multi-robot · Scheduling · Synchronized

List of Symbols

R	Set of mobile robots on the IWs
N_c	Grid area at two dimensions
O_{obs}	Barrier raster set
g	Arbitrary raster set
$T_e(x_e, y_e)$	Set of target point
$S_s(x_e, y_e)$	Starting point for mobile robot

D	Set of order need to process over a certain period
T	Set of tasks need to be carried by robots for specific order
C	Represented transportation of robot r_i
$T_{ii1} \cdot C_{ijji}(j-1)$	Transportation time of the robot r_i runs from its parking point to the shelf storage point
$W(r_i)$	Walking costs for each robot
$ITC\{r_i, T_k\}$	Total cost for the robot r_i to complete the assigned task
F_1	Longest time taken for a robot to complete its task
F_2	Total completion time for the order
V_i	Velocity vector (PSO)
X_i	Position vector (PSO)
P_g	Best location which can be found by the current group and which can be a globally optimal solution
ξ and δ	Number between 0 and 1
C_1	Self-confidence factor
C_2	Swarm confidence factor
f	Fitness function

✉ Ray Y. Zhong
 zhongzry@gmail.com

- ¹ Guangdong Provincial Key Laboratory of Computer Integrated Manufacturing Systems, School of Electromechanical Engineering, Guangdong University of Technology, Guangzhou, Guangdong, China
- ² Department of Mechatronics Engineering, Kennesaw State University, Kennesaw, USA
- ³ Department of Industrial and Manufacturing Systems Engineering, The University of Hong Kong, Pok Fu Lam, Hong Kong
- ⁴ School of Mechanical and Aerospace Engineering, Nanyang Technological University, Singapore, Singapore

xsize	Population size
maxgen	Number of iterations
w	Inertia weight
P_k^g	Position of the particle with best global fitness at current move k
pc	Crossover probability
pm	Mutation probability

Introduction

E-commerce has not only shaken the traditional trade industry within the last few years, but it has also led to increased attention being focused on the logistics industry (Alavidoost et al. 2015). A traditional logistics operation is inefficient and is usually carried out manually. Therefore, robotic and intelligent management systems were introduced to the traditional logistics and warehouse system as a means of improving the efficiency and productivity of the system (Costa et al. 2017; Liu et al. 2017; Zhong et al. 2014). In intelligent warehousing systems (IWs), the handling of objects is carried out by the robotic system such as the automated guided vehicle system (AGVs). The basic task of the warehouse system is the preservation of goods, regulation of the goods' circulation, supervision of the goods' quantity and quality, and the distribution and loading of the goods (Barenji et al. 2014). Therefore, efficient and sensitive warehousing with intelligent mobile robots becomes of utmost importance for improving the overall productivity and efficiency of the system. With facing robotic systems in IWs advanced scheduling and control system is necessary in order to improve performance. Scheduling is responsible for optimizing the mobile robots' processes in the system based on customer's demands. In other words, scheduling is responsible for the system performance; for example, when orders are received, they divide it into different batches, such as packaging, dispatching, and store. It means that scheduling in IWs faces with complex and stochastic environment, which are vital components in the development of an innovative, intelligent, and optimized scheduling system (Heragu et al. 2005). In general, the scheduling of IWs differs from other existing systems, such as manufacturing and traditional warehouse management in three ways. First, in IWs an order is accepted based on the availability and capability of equipment, at the same time advanced robotic systems are more adapted so the equipment capability of IWs is higher than other existing systems. Second, the number of orders and cancellations are higher than manufacturing enterprises, therefore the amount of warehouse data generated per unit of work is very fluctuating. Third, the demand in IWs is highly dynamic. Hence, researchers used heuristic algorithms and advanced technology for solving scheduling problems in IWs. Further, researchers have suggested that warehouse scheduling is a

deterministic multi-item inventory problem with a restricted warehouse floor availability (Barenji et al. 2017b; Chen et al. 2016; Yan et al. 2015). Hariga and Jackson (1996) tackled warehouse scheduling by employing mixed integer nonlinear programming problems, with the main objective of minimizing long-run inventory holding and order costs per unit of time.

Minner and Transchel (2010) considered the problems at the interface of marketing and operations, in order to find the optimal lot-size and selling price for multiple products with limited storage capacity in the warehouse. Gu et al. (2007) prepared a comprehensive review of the planning problems associated with warehouse operations, and problems were classified according to basic warehouse functions. These problems were divided into two main parts; namely, warehouse design problems, which can be further subcategorized into overall structure (Baker and Canessa 2009), sizing and dimensioning (Gu 2005), departmental layout (Hassan 2002) and so forth. The operational problems primarily categorized into receiving, shipping, storage and order picking (Rouwenhorst et al. 2000), all of which are considered to be scheduling and control problems (Barenji et al. 2017b; Gu et al. 2010). Chen et al. (2016) proposed an ant colony optimization (ACO)-based online routing method to find pick routes for multiple-order pickers under a non-deterministic picking time in a warehouse. Nastasi et al. (2016) presented and implemented a multi-objective optimization GA for the management of automated warehouse. Huh et al. (2017) introduced a method which aims to optimize quickly travel routes for a large-scale automated storage and retrieval systems by using case-based reasoning. Foumani et al. (2018) considered an AS/RS in which a Cartesian robot picks and palletizes hazardous items onto a mixed pallet for any order. The decision to be made includes finding the optimal sequence of orders and optimal sequence of multiple items with minimize total travel times. Ye et al. (2017) proposed an effective fireworks algorithm (FWA), which is a new heuristic algorithm inspired by the phenomenon of a fireworks display, to solve the warehouse-scheduling problem. The authors formulated warehouse scheduling based on a single-objective optimization problem without considering robot synchronization. Yalcin et al. (2018) addressed the problem of retrieving a stored unit load from puzzle-based storage using the minimum number of item moves and optimized this problem via a heuristic algorithm for big-size problem.

Ponnambalam et al. (2001) offered a multi-objective GA to derive optimal machine-wise priority dispatching rules for resolving job-shop problems with objective functions which consider the minimization of makespan, total tardiness, and total machine idle time. Contreras-Cruz et al. (2015) introduced an evolutionary method for solving the mobile robot path planning problem; the proposed approach uses

the Artificial Bee Con algorithm as a local search procedure. Zhang et al. (2013) proposed a multi-objective path planning algorithm based on particle swarm optimization (PSO) for robot navigation. First, a membership function is defined to evaluate the degree of risk for a path then considering two performance indicators, the degree of risk and path distance, the path planning problem with uncertain danger sources was described as a constrained bi-objective optimization problem with uncertain coefficients. Zhang et al. (2013) studied the robot path planning problem with danger sources by using a multi-objective PSO algorithm. However, the work focused on the danger sources with precise positions.

Adapted advanced robotic system in the IWs create a complex and more stochastic environment. This environment needs a new type of scheduling and control system which can be answer to new problems, such as robot minimum path planning, energy consumption, customer demand, and so forth. Stojanovic et al. (2016) proposed a cascade load force control design for a parallel robot platform. Authors formulated a control design problem based on an optimization problem under constraints and used a simulation method for the validation of the results. Pršić et al. (2017) presented an optimal control design for a pneumatically driven parallel robot platform. The Proportional-Integral-Derivative (PID) algorithm with feedback linearization is used for control. This search method was based on a firefly algorithm due to the empirical evidence of its superiority in solving various nonconvex problems. In addition, the optimal control of a hydraulically-driven parallel robot platform based on a firefly algorithm was proposed by Nedic et al. (2015). More precisely Zhou et al. (2014) presented a new mechanism for multi robot task allocation in IWs; the mechanism of system with an unknown task cost and then the objective of the mechanism becomes twofold i.e. equal allocation of the workload as well as the minimization of travelling expenditures. Moreover, Zhou et al. proposed a balanced heuristic mechanism for achieving a newer mechanism. In another study carried out by Dou et al. (2015), a new hybrid solution for the multi-robot problem in the intelligent warehouse was presented for improving the efficiency of the system based on the GA in order to reinforce the learning method, they focused on minimizing the planning path for the mobile robots.

The scheduling of multi-robot in IWs is limitedly reported when faced with multiple simultaneous customer demands. Whilst dynamic customer demands create the main scheduling problem on the IWs, meanwhile customer satisfaction can be improved if a system can overcome this problem (Barenji et al. 2017a). In order to solve this problem, this paper proposes a mechanism based on the PSO algorithm. In the proposed mechanism, we consider two main objectives. The first is reduce the total cost of the robot by minimizing the maximum completion time. The second is to minimize the sum of the transport synergies for all orders by consid-

ering the degree of aggregation of the completion, as well as the total completion time of the orders. Experiments are carried out to evaluate the proposed approach by comparing with a GA based method.

The main contributions of this paper are as follows: (1) a mathematical model of the system is developed, and two objectives are considered. (2) A PSO algorithm is proposed for solving the scheduling problem in terms of the two main objectives: reducing the total cost of the robot by minimizing the maximum completion time and minimizing the sum of the transport synergies for all orders by considering the degree of aggregation. (3) A coding method is introduced based on the bi-level coding of order and tasks.

The rest of this paper is organized as follows: In “**System description**” section, the system description is discussed. “**Proposed mathematical model**” section proposes a mathematical model of the system based on the literature review. Next, “**PSO algorithm based solution**” section introduces the PSO algorithm solutions for the developed mathematical model. Furthermore, “**Experiments**” section illustrates the experiments necessary for the evaluation of the proposed model. Finally, “**Conclusion**” section limitations and concludes this paper by highlighting the key findings and future research directions.

System description

When multiple orders or dynamic customer demands are send to IWs, they are usually managed by a task set and all resources, such as mobile robots for picking and placing purpose, are scheduled based on the runtime between the task points and the completion time of each task. In addition, the amount of robots that are effective is critical. Since some tasks needs to be finished on a synchronized basis, therefore it is possible to evaluated the cost of each robot based on the minimization of the maximum completion time (Dias et al. 2006).

In this paper, our main discussion concerns multi-order task scheduling and path planning for multiple-robot in IWs. In real-world warehousing, for example, the Amazon Kiva system, there are usually dozens or even hundreds of mobile robots which crawl and move the shelves to the packaging station. In order to simplify the system, we make the following assumptions which are already used in industrial system:

- (1) Each robot is equipped with the necessary sensors for detecting the environment.
- (2) Each robot can only run in four directions i.e. up, down, left and right.
- (3) The robots and tasks are identical with each other.
- (4) Robots are independent when performing tasks.

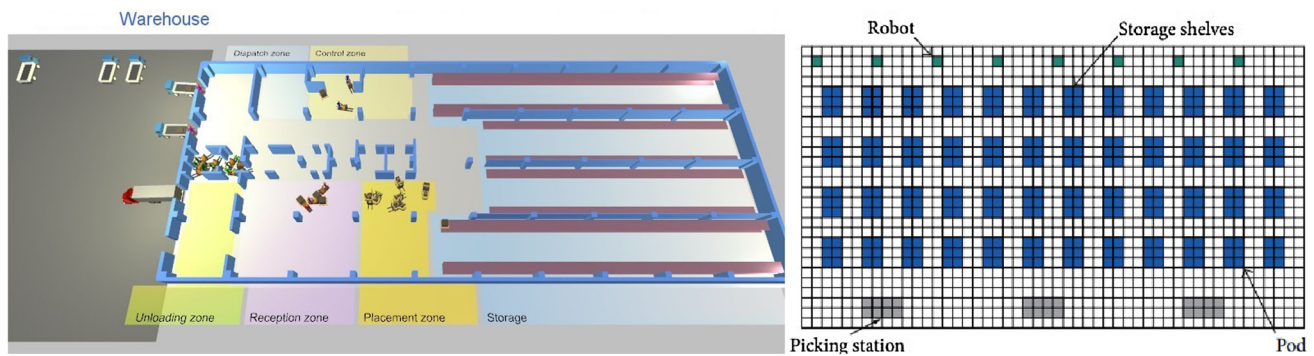


Fig. 1 3D model and rasterized simple intelligent storage environment

- (5) The maximum completion time and total completion time of a robot are measured by its movements (the shortest distance between the start and end points).
- (6) The runtime between the packaging station and the shelves are fixed.
- (7) Robots are executing tasks without considering collision.

The 3D and grid models of IWs are shown in Fig. 1. The mobile robot (R) is regarded as a moving object in a two-dimensional environment. The storage location of all shelves in the system is mapped on an inaccessible hazardous area. R moves in a finite region, where there are a number of obstructions (shelves). After rasterization, the obstacle is converted into an obstacle area. Assuming that the step size for R is u and the maximum areas on X and Y are $X_{\max}$ and $Y_{\max}$, the number of grid in a row are $N_c = X_{\max}/u$, and $Nl = Y_{\max}/u$ respectively.

As shown in Fig. 1, the obstacles are evenly distributed. R 's position at the starting and ending point is uncertain. We denote an arbitrary raster as g and the grid form a set A , with the barrier raster set denoted by $O_{obs} = (o_1, o_2, \dots, o_n)$ where $o_i \in O_{obs} (i = 1, 2, \dots, n)$. For any g that belongs to A , the corresponding coordinates in XOY are (x, y) , resulting in $g(x, y)$. Let the first grid in the bottom left corner be defined as $(1, 1)$ and let $N_{num} = \{1, 2, 3, \dots, M\}$ be a raster number set, where the serial number of $g(1, 1)$ is 1. Then $g(2, 1)$ has the serial number $(N_c + 1)$. The coordinates (x_i, y_i) and serial number $i \in N_{num}$ of g_i belongs to the set A which constitutes a mutual mapping. The coordinates of the serial number i are determined by the following formula (in which the “mod” and “int” represent the fetch and rounding operators, respectively):

$$\begin{cases} x_i = ((i - 1) \bmod N_c) + 1 \\ y_i = (\text{int})((i - 1)/N_c) + 1 \end{cases}$$

The path planning in a grid environment means that the robot arrives at any set of target point $T_e(x_e, y_e)$ from any

starting point $s_s(x_s, y_s)$ along a superior path without a collision in the area, where $S, T \notin O_{obs}$ and $s, e \in N_{num}, s \neq e$. The runtime between the two points can be estimated by $t_{se} = |x_e - x_s| + |y_e - y_s|$. For solving this scheduling problem for multiple robots in a warehouse system, we turn on the PSO algorithm. PSO is a heuristic search technique which is inspired by the behavior of flocks of birds. Although PSO is relatively new, the relative simplicity, fast convergence and population-based features have made it a viable alternative for solving scheduling problems. Moreover, existing studies have shown the competence of PSO in tackling scheduling problems with static or dynamic obstacles (Venter and Sobieszczanski-Sobieski 2003). The main advantages of the PSO algorithm can be categorized as follow (Bai 2010):

- (1) PSO was proposed and developed based on intelligence so it can support both scientific research and engineering uses.
- (2) PSO has no overlapping and mutation calculations. The search can be carried out using the speed of the particle. During the development of several generations, only the most optimist particle can transmit information to the other particles, and the researching speed is very fast.
- (3) Compared with other algorithms, it has a greater optimization ability and can be completed easily.
- (4) PSO adopts a real number code, and it is decided directly by the solution. The dimension is equal to the constant in the solution. Based on the existing advantages of the PSO algorithm, we selected this algorithm for optimizing the proposed mechanics explained in the objectives.

Proposed mathematical model

The scheduling of dynamic order while considering multiple robots in IWs is highly complex and an NP hard problem (Dou et al. 2015; Elango et al. 2011; Zhang et al. 2013), it is usually referred to as an assembly flow shop (Dou et al. 2015; Elango et al. 2011; Gautam et al. 2016). Therefore, we assume that the warehouse system has D orders $D = \{d_1, d_2, \dots, d_j\}$ to be processed over a certain period of

time, in which each order has I goods belonging to different shelves $I = \{1, 2, \dots, i\}$ which needs to be handled. Then a total of $I \times D$ tasks $T = \{t_{11}, t_{12}, \dots, t_{1i}, \dots, t_{j1}, \dots, t_{ji}\}$ (where $j \in D, i \in I$) need to be carried out by m robots $R = \{r_1, r_2, \dots, r_m\}$. We assume that the task set T is divided into m subsets, $T = \{T_1, T_2, \dots, T_m\}$, where T_i is assigned to the robot r_i , i.e. the robot r_i is assigned to k tasks, denoted by $T_{ik} = \{t_{i1}, t_{i2}, \dots, t_{ik}\}$. The transportation time of the robot r_i runs from its parking point to the shelf storage point as represented by T_{ii1} . $C_{ijji(j-1)}$ represents the transportation time of the robot r_i which runs from the $(j-i)$ th to the j th order task storage point. It should be noted that $C_{ijji(j-1)}$ is equal to $C_{ii(j-1)ij}$ in the whole system. W_{in} represents the execution time of the n th order task of the robot r_i i.e. the execution time of the robot r_i for transporting the corresponding shelf of the n th order task from the shelf storage point to the packaging table, and then returning it to the original shelf after the packaging has been processed. Therefore, the sum of the walking costs for each task T_{ik} is $W(r_i) = w_{i1} + w_{i2} + \dots + w_{ik}$. The variables T_{ii1} , $C_{ijji(j-1)}$ and W_{in} are calculated according to the actual coordinate position in the rasterized system.

Therefore, it is possible to define the total cost for the robot r_i to complete the assigned task as $ITC\{r_i, T_k\}$ can be expressed as follows:

$$ITC\{r_i, T_k\} = T_{ii1} + \sum_{j=2}^k C_{ijji(j-1)} + W(r_i) \quad (1)$$

In this paper, we set two evaluation indicators for task allocation. The first indicator is the longest time taken for a robot to complete its task. The second indicator shows the convergence degree of the completion time for all goods from an order d as well as the total completion time for the order. In order words, it is the degree of transportation synergism of the order. The specific expressions relating to these indicators are as follows:

Objective Functions:

$$F_1 = \text{Max}ITC\{r_i, T_i\} \quad (2)$$

$$F_2 = \sum_{d=1}^D \sqrt{\sum_{q=1}^{I_d} (t_{dq} - t_d)^2} \quad (3)$$

Subject to

$$\sum_{t,d} X_{pxdt} = 1, \quad \forall p, x \quad (4)$$

$$\sum_{j=1}^J N_{pmj} = 1, \quad \forall x, d \quad (5)$$

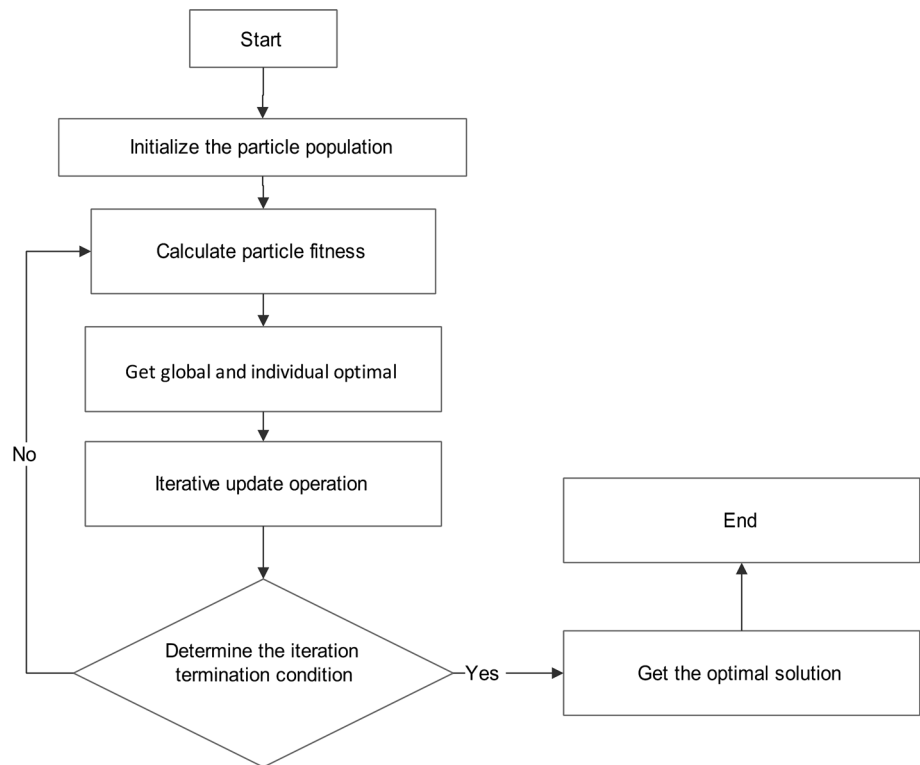
$$\sum_{j=1}^J NM_{pim}(j) = 1, \quad \forall p, m \quad (6)$$

$$ST_{px(d+1)} \geq CT_{pxd}, \quad \forall p, x, d \quad (7)$$

In these equations, r_i represents the robot i . T_i represents the task which is assigned to r_i from task set T . F_1 represents the maximum completion time. D is the number of orders in the order task set, d is $0 < d \leq D$ and q is the q th order task in the order d . I_d is the total number of order tasks in the order d , and t_{dq} indicates the completion time of the q th order task in the d th order. t_d indicates the completion time for order d . F_2 indicates the order match information. Constraint (4) imposes that the batch P at position x in time t can only belong to a job phase d . Constraint (5) indicates that the P -batch x -position goods can only be placed on a robot at the same time for handling operations. Constraint (6) imposes that the number of the handling robots in the i th procedure of the P batch item m is 1. Constraints (7) shows that the P -batch x -position and the start-up time of the item in the $d+1$ process is greater than the completion time in the d th process.

PSO algorithm based solution

The PSO is inspired by the social behavior of birds, especially their ability to locate a desirable position in a given area. Proposed first by Kennedy (2011). The PSO algorithm adopts a speed-location search model for completing iterative updates (Chen et al. 2016; Gautam et al. 2016). The search of the particles in the solution space is based on the speed-location search model as shown in Fig. 2. More precisely, a swarm of particles is used in PSO to represent the potential solutions, and each particle i is defined by two vectors, the velocity vector $V_i = [v_{i1}, v_{i2}, \dots, v_{iD}]$, and position vector $X_i = [x_{i1}, x_{i2}, \dots, x_{iD}]$, where D is defined as a D -dimensional solution. The first step is to note particle i 's random based velocity and position in the search space. During the evolutionary process, the particle i is evaluated according to its present position. If the present fitness is better than the fitness of pB_{id} , which stores the best solution which the i th particle has explored so far, then pB_{id} will be replaced by the current solution. The algorithm selects the best pB_{id} of the swarm to be the best globally and denotes as gB_d . In order to achieve a high performance, we decrease linearly the value of the inertia weight from about 0.9–0.4 during a run. The detailed information regarding encoding, the feasibility solution, and the iteration of proposed PSO are provided in the next section.

Fig. 2 PSO algorithm flow chart

Encoding

In 1997, a binary coded Discrete Particle Swarm Optimization (DPSO) was proposed and each bit of the particle position vector was assigned to be 1 or 0. In response to the need for solving the problem of Permutation Flow Shop Scheduling (PFSP) (Alavidoost et al. 2015), this paper utilized a sequential coding method to encode the particles. Therefore, a double coding (DE) method is proposed with the parts and orders encoded sequentially, respectively, in order to determine the scheduling sequence.

Suppose, for example, that the number of orders $D = 3$ and that there are 5 products in each order. If the location of a particle is $X_i = \{(2, 1, 3), (3, 2, 1, 5, 4), (1, 3, 4, 5, 2), (1, 5, 2, 3, 4)\}$, then the first three bits determine the sequence of orders. Hence, the sequence of the orders is given as (2, 1, 3). The latter three parts indicate the order's sequence of the products in each order {2.3, 2.2, 2.1, 2.5, 2.4}; {1.3, 1.1, 1.2, 1.4, 1.5}; {3.1, 3.5, 3.2, 3.3, 3.4}. $a.b$ represent the position of the b th product in the a th order. Then the first position is occupied by the third product in order 2, while the final position is occupied by the fourth product in order 3.

Feasible solution of the adjustment

In order to ensure that each position of the particles $X_i = [x_1, x_2, \dots, x_n]$, in the iteration process (i.e. when the algorithm is running the location code after each update) is still a feasi-

ble solution (a feasible scheduling scheme), we designed an adjustment strategy for the feasible solution in the process of coding design, in particular. For a better understanding of the designed strategy, we provide the following example:

Assume the number of products $n = 5$, the initial position of a randomly generated particle is $X_i = (4, 1, 5, 2, 3)$, and the particle is updated by using the iteration formula to obtain a new position $X_{i+1} = (3, 8, -2, -4, 7)$. Since the position parameter will be positive, the updated X_{i+1} is not a feasible solution. Therefore, it is necessary to adjust X_{i+1} to make it a feasible solution. The specific steps involved in the feasible solution adjustment are as follows:

- (1) Let each value which is greater than 5 in $X_{i+1} = (3, 8, -2, -4, 7)$ be set equal to 5 and then, an adjustment made such that each value less than 1 is set equal to 1, so that we can obtain the $X_{i+1} = (3, 5, 1, 1, 5)$;
- (2) The duplicated values in $X_{i+1} = (3, 5, 1, 1, 5)$ keeps their value, and the rest of the duplicate values are adjusted to 0, so that $X_{i+1} = (3, 0, 1, 0, 5)$;
- (3) to randomly generate a feasible solution $X_{i+2} = (2, 5, 3, 1, 4)$ with the same dimension as X_{i+1} , integers between 1 and 5 are generated randomly for each position. The values which X_{i+1} and X_{i+2} share is set equal to 0 in X_{i+2} . The rest of the values remain unchanged. Hence $X_{i+1} = (3, 0, 1, 0, 5)$;
- (4) Next the zero values in $X_{i+1} = (3, 0, 1, 0, 5)$ are replaced by the nonzero values in $X_{i+2} = (2, 0, 0, 0, 4)$. The

Table 1 A particle code table

	Order					Order 1					Order 2					Order 3				
Serial number	1	2	3	1	2	3	4	5	1	2	3	4	5	1	2	3	4	5		
Position	2	3	1	3	1	4	5	2	2	1	4	3	5	4	2	3	5	1		

Table 2 Particle order and part arrangement table

From left to right order																			
32	35	31	33	34	12	11	14	13	15	25	22	23	21	24					

adjusted feasible solution is given as $X_{i+1} = (3, 2, 1, 4, 5)$.

- 5) After the iteration of the particle $X_i = (4, 1, 5, 2, 3)$, the new particle is $X_{i+1} = (3, 2, 1, 4, 5)$. Therefore solution is now feasible.

Iterative update

In the proposed algorithm, the particle needs to be evolved by utilizing the iterative update operation (Poli 2008). The iteration update formula for a particle's velocity and position are as shown in Eqs. (8) and (9).

$$V_m^{n+1} = wV_m^n + c_1\xi(P_m^n - X_m^n) + c_2\delta(P_g^n - X_m^n) \quad (8)$$

$$X_m^{n+1} = (X_m^n + V_m^{n+1}) \quad (9)$$

In the above formulas, $P_g = (P_{g1}, P_{g2}, \dots, P_{gN})$ refers to the best location which can be found by the current group and which can be a globally optimal solution. The optimal solution and the global solution are determined by the fitness value which corresponds to the particle. Next c_1 and c_2 are the acceleration factor (also known as the learning factor). Here, w is the inertia weight. ξ and δ are random numbers between 0 and 1. $V_m^n = (V_{m1}^n, V_{m2}^n, \dots, V_{mN}^n)$ represents the velocity of the m th particle at the n th iteration. This is a vector of N dimensions, and each element of the vector represents the velocity of the dimension. $P_m^n = (P_{m1}^n, P_{m2}^n, \dots, P_{mN}^n)$ represents the current best position found by the m th particle itself. That is the optimal solution of the individual. $X_m^n = (X_{m1}^n, X_{m2}^n, \dots, X_{mN}^n)$ represents the position for the m th particle at the n th iteration, which corresponds to the scheduling scheme of the PFSP. X_{mi}^n represents the processing order number of the i th workpiece, which is to be processed by the m th particle at the n th iteration.

Decoding and multi-objective data normalization

In PSO, each particle has a certain coding which is used for determining the sequence of the orders and products. We

Table 3 The distribution matrix of a particle on a robot

	Order handling arrangement matrix (from left to right)				
The 1st robot	32	33	11	15	23
The 2nd robot	35	34	14	25	21
The 3rd robot	31	12	13	22	24

can decode the identified code into the product delivery array matrix. According to a scheduling scheme, the corresponding objective function value can be obtained. At the same time, the PSO algorithm is iteratively updated according to the objective function value.

An example is given to illustrate the generation of a product delivery array matrix. Assuming that there are three orders, three robots, and each order contains five products as seen in Table 1.

The orders are arranged in the order of 3–1–2. The sequences of parts in order 1, 2, 3 are {2, 5, 1, 3, 4}, {2, 1, 4, 3, 5}, {5, 2, 3, 1, 4}. The sequences of all parts can be seen in Table 2.

Each unit (ab) in Table 2 represents the b th product of the a th order; the handling sequences of the products arranged to three robots can be seen in Table 3.

In the algorithm, when the particles are decoded and the fitness is calculated according to the scheduling scheme, our objective functions becomes F_1 and F_2 ; these functions represents the degree of simultaneity and punctuality, respectively. In the algorithm for the fitness design, we need to transform the multi-objective optimization problem into a single-objective optimization problem, and then, solve the problem by using single-objective optimization. In this paper, the linear weighing method is used for the transformation, in which the objective function is $\min F = F_1 + F_2$. It can be found from the experimental test that F_1 and F_2 do not have the proper order of magnitude and need to be standardized. Standardization formula is shown below.

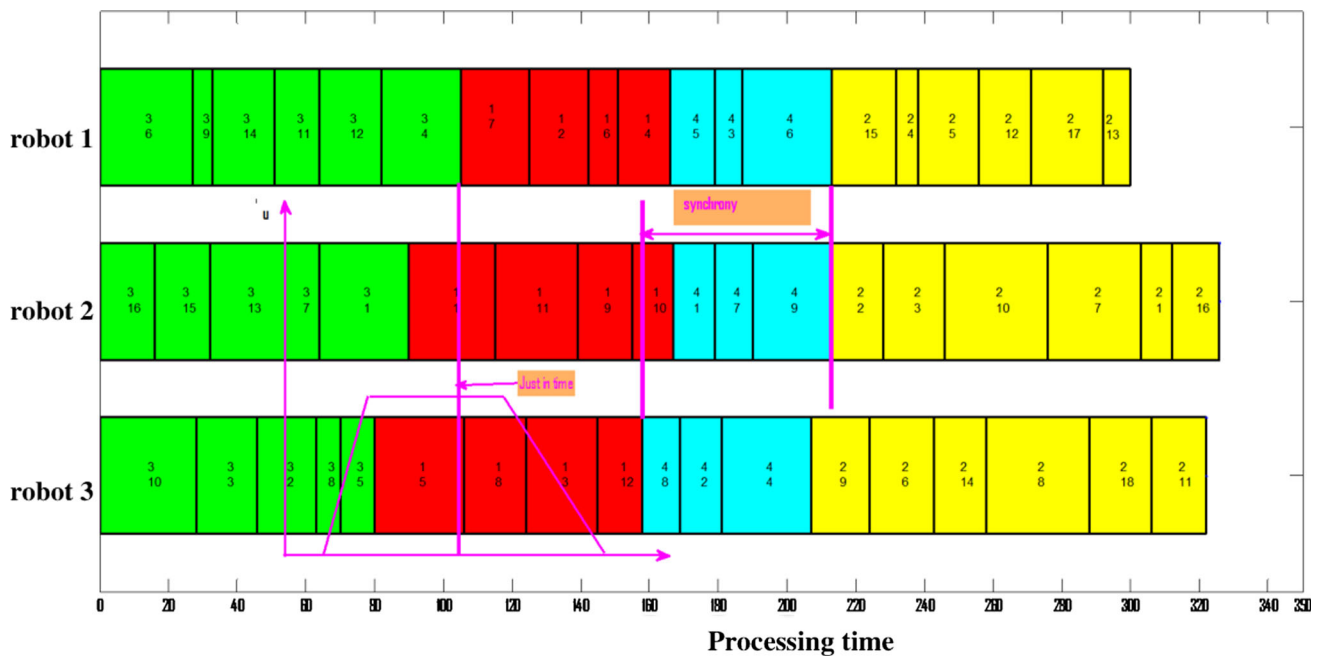


Fig. 3 PSO algorithm scheduling Gantt chart

$$v'(i) = \frac{v(i) - \min[v(i)]}{\max[v(i)] - \min[v(i)]} \quad (10)$$

After the standardization, the target function becomes $\min F = F'_1 + F'_2$, where F'_1 and F'_2 are the standardized values of F_1 and F_2 , respectively. In the proposed algorithm we consider the problems in which the particle swarm scale is defined as 200. The length of particle D is assigned to be 2 based on the Schaffer function. $V_{\max}$ is defined to be the maximum movement distance of the particle in one circulation and set as the range width of the particle. C_1, C_2 is assigned to be 2. The conditions include the maximum circulation time and the minimum warp requirement, and the maximum circulation time is the terminal condition of the article.

Experiments

In this section, we test the performance of the proposed model by simulating an intelligent storage system environment. This is done by mapping the warehouse to a 50×28 rasterized two-dimensional plane space, in which each robot and shelf occupy a grid. Afterwards, a total of 4×12 small storage areas constitute the barrier area in the center. Each small storage area consists of six shelves. The sorting station is located at the bottom of the system. We assumed that three robots serve the storage system and, each robot can only move within a grid range. The optimization performance of the PSO algorithm depends largely on the control parameters, namely, the population size (xsize), number of iterations

(maxgen), inertia weight (w), and the acceleration coefficient (c_1, c_2). In the experimental design of the algorithm, the control parameters were $c_1/c_2 = 2-2$, $w = 0.7$, $xsize = 50$, and $maxgen = 200$. We assumed that there were 4 orders arriving at the same time, and the number of tasks per order were 12, 18, 16, and 9. After the optimization by PSO algorithm, the scheduling results can be seen on the Gantt chart (Fig. 3), which reflects the order of execution of the order task, where the different colors represents the different orders. Each unit represents the execution time and order of an individual task. In each unit, the upper values represents the order numbers and the bottom values represent the task serial numbers. It can be seen that the orders can be completed with synergy and timeliness.

Comparative analysis of PSO and GA

In order to validate the proposed approach, this section reports on a comparative study of the PSO and GA algorithms. GA is meta-heuristically inspired by the process of natural selection which belongs to the larger class of evolutionary algorithms (EA) and is used commonly to create high-quality results in optimization and search problems. The GA has been popular in academia and industry mainly because of its intuitiveness, ease of implementation, and its ability to solve effectively the highly nonlinear and mixed integer optimization problems which are typical of complex engineering systems. The limitation of the GA is its expensive computational cost. Therefore, many researchers have attempted to examine that the PSO is as same effectiveness

Table 4 Comparison of PSO algorithm and GA algorithm results

Problem	N * M	F F1 1/F2	PSO			GA		
			BRE	ARE	WRE	BRE	ARE	WRE
Care1	4 * 3	0	0	2.1e−6	2.0e−5	0	0.0001	0.0014
		326.4	0	27.2	51.7	33.8	161.0	272.1
		0.2776	0	0.0048	0.0127	0.014	0.1636	0.4346
Care2	4 * 4	0.0173	0	0.017	0.043	− 0.007	0.0231	0.0650
		257.9	0	27.1	51.9	47.1	150.7	399.1
		0.2881	0	0.0057	0.01	− 0.112	0.088	0.180
Care3	4 * 5	0.0055	0	0.0330	0.0653	− 0.005	0.0461	0.0918
		235.1	0	15.0	42.1	30.3	92.8	145.9
		0.318	0	0.0034	0.0057	0.005	0.039	0.100
Care4	6 * 3	0	0	0.0045	0.0182	0	0.0492	0.0795
		833.9	0	42.2	86.5	54.2	302.8	708.3
		0.199	0	0.0041	0.0112	0.028	0.056	0.086
Care5	6 * 4	0.0146	0	0.0395	0.0772	− 0.014	0.0621	0.1380
		715.5	0	26.6	61.4	85.1	210.6	433.0
		0.249	0	0.0053	0.0094	0.0174	0.0522	0.0730
Care6	6 * 5	0.1050	0	0.0272	0.0665	0.017	0.0691	0.1486
		573.3	0	51.8	111.7	73.3	232.9	411.5
		0.3155	0	0.006	0.0161	− 0.016	0.041	0.092
Care7	8 * 3	0	0	0.0018	0.0135	0	0.0133	0.0368
		1034	0	45.2	75.3	130.8	350.8	608.5
		0.1572	0	0.0038	0.0089	0.0134	0.0611	0.1451
Care8	8 * 4	0	0	0.0041	0.0103	0	0.0158	0.0316
		883.8	0	38.5	68.6	61.5	356.8	690.2
		0.1901	0	0.0058	0.016	0.0291	0.0938	0.3231
Care9	8 * 5	0	0	0.0058	0.0197	0	0.0215	0.0486
		833.4	0	45.2	117.8	183.5	380.5	735.1
		0.193	0	0.0061	0.0124	0.0305	0.0951	0.1428
Care10	12 * 3	0	0	0	0	0	0	0
		1411.4	0	63.2	126.3	213.7	613	1060
		0.1012	0	0.0075	0.0262	0.0376	0.0671	0.1047
Care11	12 * 4	0	0	0	0	0	0.0011	0.0107
		1099	0	87.2	169.1	173.2	384.7	735.6
		0.1135	0	0.0072	0.0218	0.0311	0.0614	0.1102
Care12	12 * 5	0	0	0	0	0	0.0073	0.0369
		968.4	0	56.6	138.5	129.1	431.1	874.1
		0.124	0	0.006	0.014	0.019	0.052	0.091

(in finding the true globally optimal solution) as the GA but with better computational efficiency (fewer function evaluations) due to implementing statistical analysis and formal hypothesis testing (Onwubolu and Mutingi 2003).

Like the PSO, GA begins its search from a randomly generated population of designs which evolve over successive generations (iterations), eliminating the need for a user-supplied starting point. To perform its optimization-like

process, GA employs three operators to propagate its population from one generation to another. In this algorithm, in the initial operation, the “Selection” operator mimics the principal of fitness for each chromosome. After the selection operation, the “Crossover” operator mimics mating in biological populations. The crossover operator propagates the features of good survival designs from the current population into the future population, which will have a better

Table 5 PSO algorithm and GA algorithm time comparison

Problem	N * M	GA (s)	PSO (s)
Care1	4 * 3	81.2	66.57
Care2	4 * 4	83.1	66.5
Care3	4 * 4	86.8	74.2
Care4	6 * 3	141.6	111.0
Care5	6 * 4	136.7	94.5
Care6	6 * 5	130.2	102.3
Care7	8 * 3	154.7	126.7
Care8	8 * 4	175.2	135.7
Care9	8 * 5	172.1	135.9
Care10	12 * 3	225.4	185.3
Care11	12 * 4	240.4	188.6
Care12	12 * 5	250.7	189.7

fitness value on average. The last operator is “Mutation”, which promotes diversity in the population characteristics. The mutation operator allows for a global search of the design space and prevents the algorithm from getting trapped in local minima. Most researchers (Hassan et al. 2005; Kulkarni and Venayagamoorthy 2011; Mohammed et al. 2008) have suggested that for the comparison of GA with the PSO, it is better to consider 20–100 chromosomes and small mutations. In this study we used fitness proportionate selection for selecting potentially useful solutions for recombination and binary encoding method, uniform crossover utilized and bits are randomly copied from the first or from the second parent also bit inversion used for mutation operation.

In this test population, the size “xsize” = 50, the number of iterations “maxgen” = 200, inertia weight “w” = 0.4~0.9, and the acceleration factor “c₁” = “c₂” = 2 for the PSO. For the GA, population size = 50, the number of iterations = 200, the crossover probability “pc” = 0.7, and the mutation probability “pm” = 0.2. For each combination of the parameters the GA is initialed for 16 different random initial populations as is the PSO. These 16 populations are different for each combination. Thus, in total, the GA runs 900 times.

This simulation was performed on a desktop computer with a CPU frequency of 3.30 GHz as well as the use of an 8.00-GBRAM memory by using MATLAB (R2012a). In order to verify the performance and efficiency of the PSO algorithm, the comparison was divided into the index of the optimized performance and the optimization efficiency index. The test results are shown in Tables 4 and 5.

From Tables 4 and 5, for the problem with reference values, the quantitative test results as well as the number of indexes for evaluating the performance of the results are stated below:

- Calculate the optimal relative error: The algorithm runs 10 times with the relative error between the optimal value of 10 runs and the reference optimal value; this relative error is called the optimal relative error BRE;
- Calculate the average relative error: The algorithm runs 10 runs (Ibid); the relative error between the average value of 10 times and the reference optimal value is called the average relative error;
- Calculate the worst relative error: The algorithm runs 10 times; the relative error between the worst value for 10 runs and the reference optimal value is called the worst relative error WRE.

We take the computational time for solving the problem as an index for evaluating the efficiency of the algorithm. For this paper, 10 tests on 12 sets of problems are conducted in order to calculate the three performance indicators. The test results are shown in Table 4. We take the optimal solution of the test of the PSO algorithm as the theoretical optimal solution, hence the BRE value of the PSO is 0. When the PSO corresponds to the position, the value of the three indexes becomes smaller than those from the GA-based solutions; this demonstrates that the PSO is far more superior to the GA. From Table 4, we can see that in most cases, the three indexes of the PSO are smaller than GA. From the runtime for the PSO and the GA for each set of problems, it can be seen in Table 5, the runtime of the PSO is less than the GA. Furthermore, the optimization efficiency of the PSO was higher than GA.

Conclusion

This paper focused mainly on the multitask scheduling optimization of an intelligent storage system when multiple orders were received simultaneously. When multitasking on the same order failed to reach the packaging station within the minimum time interval which lead to the backlog of the orders. To tackle this problem, the PSO algorithm was proposed for assigning the task set. We assigned two parameters in a task allocation to the evaluation index: the first one reduces the total cost of the robot in the sense of minimizing the maximum completion time, while the second one considers the degree of aggregation for the completion of all tasks for the same order as well as the total completion time of the order. Experiments have shown that the program can solve effectively the backlog of orders and tasks in order to improve the efficiency of the storage system.

Several contributions are made. Firstly, A mathematical model was established for the problem which considered the same order which failed to reach the packaging station in a minimum time interval leading to a backlog in the orders being processes in the intelligent storage system. Secondly,

according to the model, this paper proposed a coding method based on the bi-level coding of order and tasks for the same order. Simulation experiment and comparative analysis were carried out in this respect. Thirdly, by employing the use of the proposed model and associated solution, the total cost of the robot in the sense of minimizing the maximum completion time was reduced. The sum of the transport synergies for all orders were minimized by considering the degree of aggregation for completing all the tasks for the same order as well as the total completion time of the orders.

The main limitation of this study on the mathematical model we are not consider collision between mobile robots which is happened on the real environment. Also in the case study we consider estimation of the runtime between two missions. In the reality, considerate is difficult in the ad hoc event. Finally, robot break-down is ignored on the system which is happen on the real environment.

Although this study addressed several challenging issues, further work is still required. Firstly, the mathematical model could be extended according to different constraints, such as limited buffer scheduling and synergy with time window constraints. Secondly, the estimation of the runtime between the two missions. Thirdly collision between mobile robots on the system will be considered. Finally, data-driven models could be investigated if a large number of sensors are deployed in the intelligent warehouse system to capture an enormous amount of data.

Acknowledgements This work was supported by the National Natural Science Foundation of China (51405089), the Science and Technology Planning Project of Guangdong Province (2015B010131008) and China Postdoctoral Science Foundation under (2018M633008).

References

- Alavidoost, M., Tarimoradi, M., & Zarandi, M. F. (2015). Bi-objective mixed-integer nonlinear programming for multi-commodity triechelon supply chain networks. *Journal of Intelligent Manufacturing*, 29, 1–18.
- Bai, Q. (2010). Analysis of particle swarm optimization algorithm. *Computer and Information Science*, 3(1), 180.
- Baker, P., & Canessa, M. (2009). Warehouse design: A structured approach. *European Journal of Operational Research*, 193(2), 425–436.
- Barenji, A. V., Barenji, R. V., Roudi, D., & Hashemipour, M. (2017a). A dynamic multi-agent-based scheduling approach for SMEs. *The International Journal of Advanced Manufacturing Technology*, 89(9–12), 3123–3137.
- Barenji, R. V., Barenji, A. V., & Hashemipour, M. (2014). A multi-agent RFID-enabled distributed control system for a flexible manufacturing shop. *The International Journal of Advanced Manufacturing Technology*, 71(9–12), 1773–1791.
- Barenji, R. V., Ozkaya, B. Y., & Barenji, A. V. (2017b). Quantifying the advantage of a kitting system using Petri nets: A case study in Turkey, modeling, analysis, and insights. *The International Journal of Advanced Manufacturing Technology*, 93(9–12), 3677–3691.
- Chen, F., Wang, H., Xie, Y., & Qi, C. (2016). An ACO-based online routing method for multiple order pickers with congestion consideration in warehouse. *Journal of Intelligent Manufacturing*, 27(2), 389–408.
- Contreras-Cruz, M. A., Ayala-Ramirez, V., & Hernandez-Belmonte, U. H. (2015). Mobile robot path planning using artificial bee colony and evolutionary programming. *Applied Soft Computing*, 30, 319–328.
- Costa, A., Cappadonna, F. A., & Fichera, S. (2017). A hybrid genetic algorithm for minimizing makespan in a flow-shop sequence-dependent group scheduling problem. *Journal of Intelligent Manufacturing*, 28(6), 1269–1283.
- Dias, M. B., Zlot, R., Kalra, N., & Stentz, A. (2006). Market-based multirobot coordination: A survey and analysis. *Proceedings of the IEEE*, 94(7), 1257–1270.
- Dou, J., Chen, C., & Yang, P. (2015). Genetic scheduling and reinforcement learning in multirobot systems for intelligent warehouses. *Mathematical Problems in Engineering*, 2015, 1–10.
- Elango, M., Nachiappan, S., & Tiwari, M. K. (2011). Balancing task allocation in multi-robot systems using K-means clustering and auction based mechanisms. *Expert Systems with Applications*, 38(6), 6486–6491.
- Foumani, M., Moeini, A., Haythorpe, M., & Smith-Miles, K. (2018). A cross-entropy method for optimising robotic automated storage and retrieval systems. *International Journal of Production Research*, 56, 1–23.
- Gautam, A., Thakur, A., Dhanania, G., & Mohan, S. (2016). A distributed algorithm for balanced multi-robot task allocation. 2016 11th International conference on paper presented at the industrial and information systems (ICIIS).
- Gu, J. (2005). *The forward reserve warehouse sizing and dimensioning problem*. Atlanta: Georgia Institute of Technology.
- Gu, J., Goetschalckx, M., & McGinnis, L. F. (2007). Research on warehouse operation: A comprehensive review. *European Journal of Operational Research*, 177(1), 1–21.
- Gu, J., Goetschalckx, M., & McGinnis, L. F. (2010). Research on warehouse design and performance evaluation: A comprehensive review. *European Journal of Operational Research*, 203(3), 539–549.
- Hariga, M. A., & Jackson, P. L. (1996). The warehouse scheduling problem: Formulation and algorithms. *IIE Transactions*, 28(2), 115–127.
- Hassan, M. M. D. (2002). A framework for the design of warehouse layout. *Facilities*, 20(13/14), 432–440.
- Hassan, R., Cohanin, B., De Weck, O., & Venter, G. (2005). A comparison of particle swarm optimization and the genetic algorithm. Paper presented at the 46th AIAA/ASME/ASCE/AHS/ASC structures, structural dynamics and materials conference.
- Heragu, S. S., Du, L., Mantel, R. J., & Schuur, P. C. (2005). Mathematical model for warehouse design and product allocation. *International Journal of Production Research*, 43(2), 327–338.
- Huh, J., Chae, M.-J., Park, J., & Kim, K. (2017). A case-based reasoning approach to fast optimization of travel routes for large-scale AS/RSs. *Journal of Intelligent Manufacturing*, 28, 1–14.
- Kennedy, J. (2011). Particle swarm optimization. In C. Sammut, & G. I. Webb (Eds.), *Encyclopedia of machine learning* (pp. 760–766). Berlin: Springer.
- Kulkarni, R. V., & Venayagamoorthy, G. K. (2011). Particle swarm optimization in wireless-sensor networks: A brief survey. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 41(2), 262–267.
- Liu, M., Ma, J., Lin, L., Ge, M., Wang, Q., & Liu, C. (2017). Intelligent assembly system for mechanical products and key technology based on internet of things. *Journal of Intelligent Manufacturing*, 28(2), 271–299.

- Minner, S., & Transchel, S. (2010). Periodic review inventory-control for perishable products under service-level constraints. *OR Spectrum*, 32(4), 979–996.
- Mohammed, A. W., Sahoo, N. C., & Geok, T. K. (2008). Solving shortest path problem using particle swarm optimization. *Applied Soft Computing*, 8(4), 1643–1653.
- Nastasi, G., Colla, V., Cateni, S., & Campigli, S. (2016). Implementation and comparison of algorithms for multi-objective optimization based on genetic algorithms applied to the management of an automated warehouse. *Journal of Intelligent Manufacturing*, 27, 1–13.
- Nedic, N., Stojanovic, V., & Djordjevic, V. (2015). Optimal control of hydraulically driven parallel robot platform based on firefly algorithm. *Nonlinear Dynamics*, 82(3), 1457–1473.
- Onwubolu, G. C., & Mutingi, M. (2003). A genetic algorithm approach for the cutting stock problem. *Journal of Intelligent Manufacturing*, 14(2), 209–218.
- Poli, R. (2008). Analysis of the publications on the applications of particle swarm optimisation. *Journal of Artificial Evolution and Applications*. <https://doi.org/10.1155/2008/685175>.
- Ponnambalam, S., Ramkumar, V., & Jawahar, N. (2001). A multi-objective genetic algorithm for job shop scheduling. *Production Planning & Control*, 12(8), 764–774.
- Pršić, D., Nedić, N., & Stojanović, V. (2017). A nature inspired optimal control of pneumatic-driven parallel robot platform. *Proceedings of the Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science*, 231(1), 59–71.
- Rouwenhorst, B., Reuter, B., Stockrahm, V., van Houtum, G.-J., Mantel, R., & Zijm, W. H. (2000). Warehouse design and control: Framework and literature review. *European Journal of Operational Research*, 122(3), 515–533.
- Stojanovic, V., Nedic, N., Prsic, D., Dubonjic, L., & Djordjevic, V. (2016). Application of cuckoo search algorithm to constrained control problem of a parallel robot platform. *The International Journal of Advanced Manufacturing Technology*, 87(9–12), 2497–2507.
- Venter, G., & Sobieszczanski-Sobieski, J. (2003). Particle swarm optimization. *AIAA Journal*, 41(8), 1583–1589.
- Yalcin, A., Koberstein, A., & Schocke, K.-O. (2018). An optimal and a heuristic algorithm for the single-item retrieval problem in puzzle-based storage systems with multiple escorts. *International Journal of Production Research*, 29, 1–23.
- Yan, B., Yan, C., Long, F., & Tan, X.-C. (2015). Multi-objective optimization of electronic product goods location assignment in stereoscopic warehouse based on adaptive genetic algorithm. *Journal of Intelligent Manufacturing*, 26, 1–13.
- Ye, S., Ma, H., Xu, S., Yang, W., & Fei, M. (2017). An effective fireworks algorithm for warehouse-scheduling problem. *Transactions of the Institute of Measurement and Control*, 39(1), 75–85.
- Zhang, Y., Gong, D.-W., & Zhang, J.-H. (2013). Robot path planning in uncertain environment using multi-objective particle swarm optimization. *Neurocomputing*, 103, 172–185.
- Zhong, R. Y., Huang, G. Q., Dai, Q. Y., & Zhang, T. (2014). Mining SOTs and dispatching rules from RFID-enabled real-time shopfloor production data. *Journal of Intelligent Manufacturing*, 25(4), 825–843.
- Zhou, L., Shi, Y., Wang, J., & Yang, P. (2014). A balanced heuristic mechanism for multirobot task allocation of intelligent warehouses. *Mathematical Problems in Engineering*, 2014, 15–25.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.